

Data Structure and Algorithm

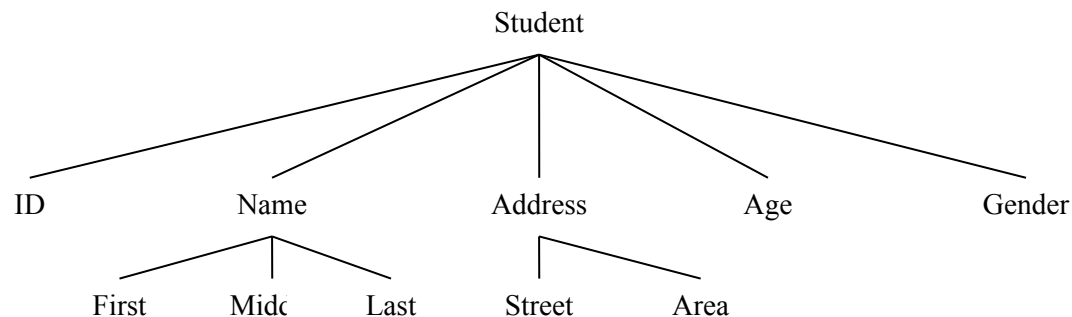
(PMSCS 623P)

Lecture 1

Data Structures

□ Data

- Data are **simply values** or sets of values, **raw materials** used as inputs.
- A data item refers to a single unit of values.
- Data items that are divided into sub items are **group items**. Those that are not are called **elementary items**. For example, a student's name may be divided into three sub items – [first name, middle name and last name] but the ID of a student would normally be treated as a single item.

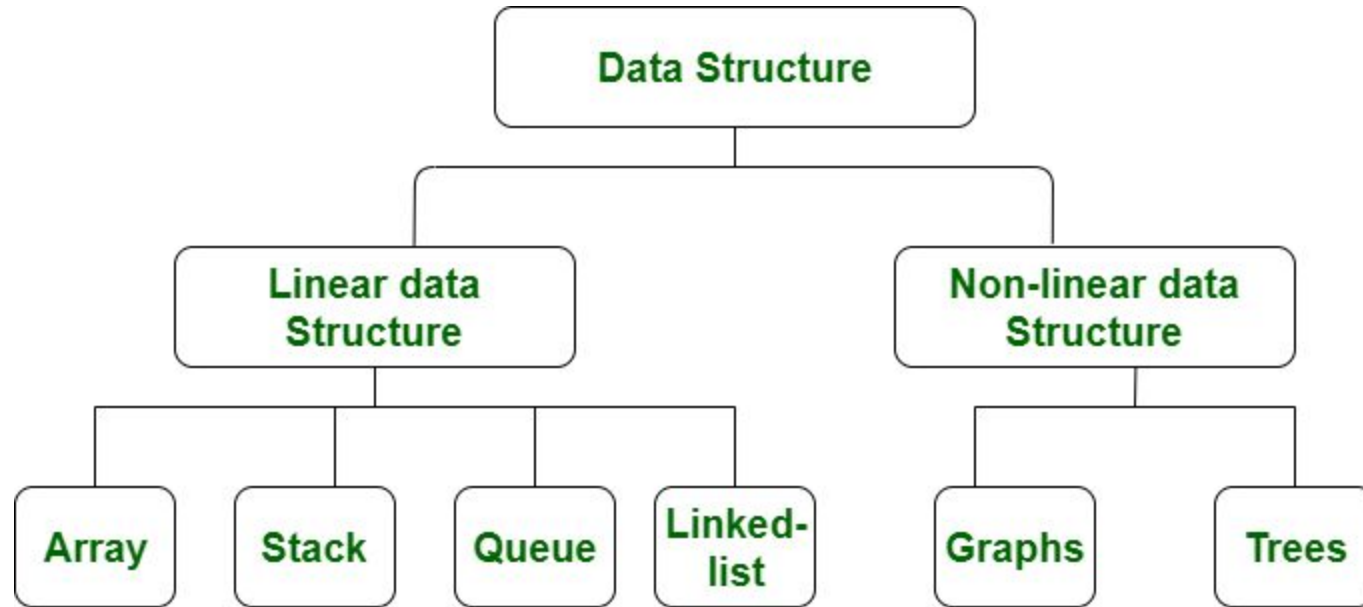


Data Structures

❑ What is Data Structure?

- A data structure is defined as a particular way of storing and organizing data in our devices to use the data efficiently and effectively. The main idea behind using data structures is to minimize the time and space complexities. An efficient data structure takes minimum memory space and requires minimum time to execute the data.

Classification of Data Structure



Classification of Data Structure

❑ Linear Data structure:

- Data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure.

Example: Array, Stack, Queue, Linked List, etc.

Classification of Data Structure

❑ Non-Linear Data Structure:

- Data structures where data elements are **NOT placed sequentially or linearly** are called non-linear data structures.
- This structure is mainly used to represent data containing a **hierarchical relationship between elements.**

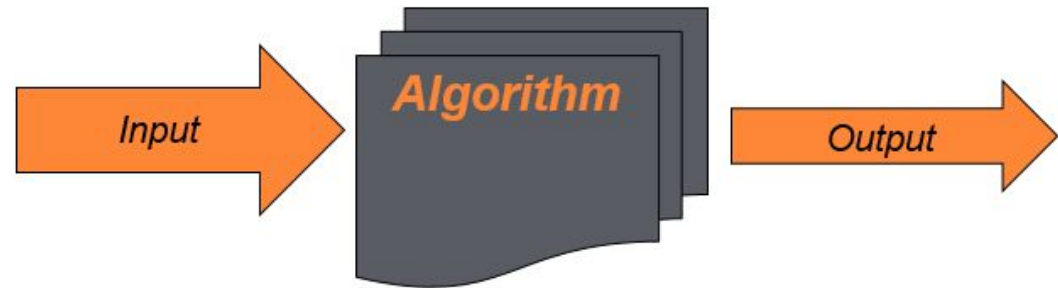
Examples: Trees and Graphs.

What is an algorithm?

- A computational procedure that takes
 - ❑ some value, or set of values, as *input*
 - ❑ produces some value, or set of values, as *output*

- ❑ may be specified:

- In English
- As a pseudocode
- As a computer program



- A **sequence** of computational steps that transform the input into the output.

What is an algorithm?

- **Algorithm** are the systematic **ordered** logical approach which is a well-defined, step-by-step procedure that allows a computer to solve a problem.
- **Algorithm** must
 - It must produce the **correct** result
 - It must **finish** in some finite time
- Algorithms are the ideas behind computer programs.
- An algorithm is the thing that stays the same whether the program is in Pascal running on a Windows or is in JAVA running on a Macintosh!

Algorithm

- Examples

Problem	Input	Output
Checking if a number is prime	A number	Yes/No
Finding a shortest path between your hostel and your department	IITG Map, your hostel name, your dept name	Well-defined shortest path
Searching an element in an array of numbers	An array of numbers	Array index

Algorithm

- **Linear search algorithm**

1. Start from the leftmost element of `arr[]` and one by one compare `x` with each element of `arr[]`.
2. If `x` matches with an element, return the index.
3. If `x` doesn't match with any of elements, return `-1`.

Pseudocode

- High-level description of an algorithm
- More structured than English prose
- Less detailed than a program
- Preferred notation for describing algorithms
- Hides program design issues

Example: find max element of an array

Algorithm *arrayMax*(A, n)

Input array A of n integers

Output maximum element of A

$currentMax \leftarrow A[0]$

for $i \leftarrow 1$ to $n - 1$ do

 if $A[i] > currentMax$ then

$currentMax \leftarrow A[i]$

return $currentMax$

Pseudocode

- It is one of the methods which can be **used to represent an algorithm for a program.**
- Many time algorithms are presented using pseudocode since they **can be read and understood by programmers** who are familiar with different programming languages.
- Pseudocode allows us to include several control structures such as **While, If-then-else, Repeat-until, for and case**, which is present in many high-level languages.
- **Note:** Pseudocode is **not** an actual programming language.

Pseudocode

```
FUNCTION linearSearch(list, searchTerm):  
    FOR index FROM 0 -> length(list):  
        IF list[index] == searchTerm THEN  
            RETURN index  
        ENDIF  
    ENDLOOP  
    RETURN -1  
END FUNCTION
```

Program

- A program is a set of **ordered** instructions for the computer to follow.
- The machine can't read a program directly, because it only understands machine code. But we can write stuff in a computer language, and then a compiler or interpreter can make it understandable to the computer.

Program

```
// C++ code for linearly search x in arr[]. If x
// is present then return its location, otherwise
// return -1
int search(int arr[], int n, int x)
{
    int i;
    for (i = 0; i < n; i++)
        if (arr[i] == x)
            return i;
    return -1;
}
```

Algorithm vs Pseudocode vs Program

Algorithm

- An algorithm is defined as a **well-defined sequence of steps** that provides a solution for a given problem.
- Algorithms are generally **written in a natural language** or plain English language.

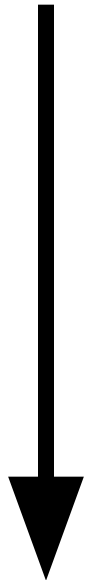
Pseudocode

- Pseudocode is one of the methods that can be **used to represent an algorithm**.
- Pseudocode is written in a format that is **similar to the structure of a high-level programming language**.

Program on the other hand allows us
to write a code in a particular programming language.

How to express algorithms?

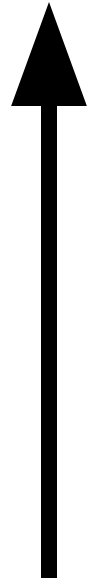
Increasing precision



English

Pseudocode

Real programming languages



Ease of expression

The Problem of Sorting

Input: sequence $\langle a_1, a_2, \dots, a_n \rangle$ of numbers.

Output: permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$ such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Example:

Input: 8 2 4 9 3 6

Output: 2 3 4 6 8 9

Insertion Sort

- ❑ Commonly used by card players: As each card is picked up, it is placed into the proper sequence in their hand.
- ❑ Divide the list into a **sorted sublist** and an **unsorted sublist**.
- ❑ In each pass, one or more pieces of data are removed from the unsorted sublist and inserted into their correct position in a sorted sublist.



$A[0]$ unused, valid
elements: $A[1] \dots$
 $A[n]$

Algorithm Name with parameters
(like a C function-header)

Comment

Pseudocode

Equivalent CPP function

INSERTION-SORT (A, n)

$\triangleright A[1 \dots n]$

Algorithm body

For loop body

```

for  $j \leftarrow 2$  to  $n$ 
do
     $\triangleright$  Insert  $A[j]$  into the sorted subarray
     $\triangleright$  in such a position that  $A[1..j]$  becomes sorted
     $key \leftarrow A[j]$ 
     $i \leftarrow j - 1$ 
    while  $i > 0$  and  $A[i] > key$ 
    do
         $A[i+1] \leftarrow A[i]$ 
         $i \leftarrow i - 1$ 
     $A[i+1] \leftarrow key$ 
  
```

While Loop body

```

void insertionSort (int A[], int n)
{
    //here  $A[0 \dots n]$  is an int array
    int i, j;
    for (j = 2; j <= n; j++) {
        key = A[j];
        i = j - 1;
        while(i > 0 && A[i] > key){
            A[i+1] = A[i];
            i = i - 1;
        } //while
        A[i+1] = key;
    } //for
}
  
```

Indentation/spacing determines where
the algorithm/loop/if/else-body ends

Insertion Sort Simulation

INSERTION-SORT (A, n) $\triangleright A[1 \dots n]$

for $j \leftarrow 2$ **to** n

do $\triangleright$ Insert $A[j]$ into the sorted subarray $A[1..j-1]$
 $\triangleright$ in such a position that $A[1..j]$ becomes sorted

$key \leftarrow A[j]$

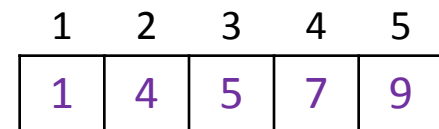
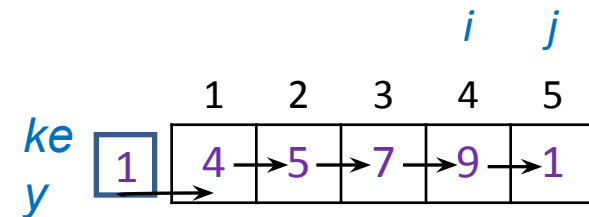
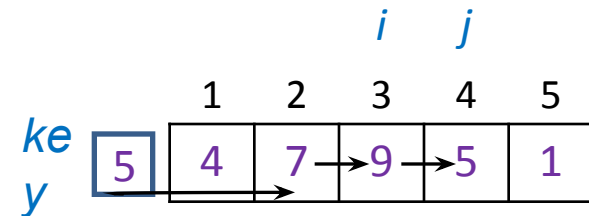
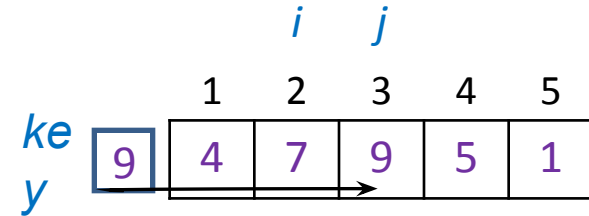
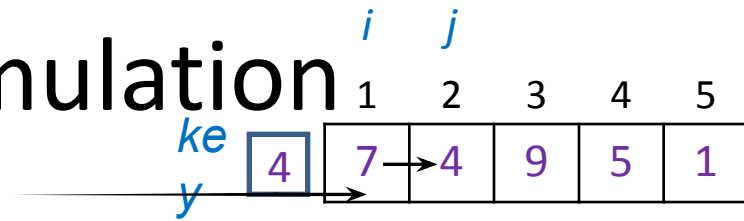
$i \leftarrow j - 1$

while $i > 0$ and $A[i] > key$

do $A[i+1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i+1] \leftarrow key$



Analysis of Algorithms

- What does it mean to analyze an algorithm?
 - To have an estimate about how much time an algorithm may take to finish, or in other words, to analyze its **running time (aka. Time complexity)**
 - Sometimes, instead of running time, we are interested in **how much memory/space** the algorithm may consume while it runs (space complexity)
 - It enables us to compare between two algorithms
- What do we mean by running time analysis?
 - Also referred to as time-complexity analysis
 - To determine how running time increases as the **size of the problem** increases.

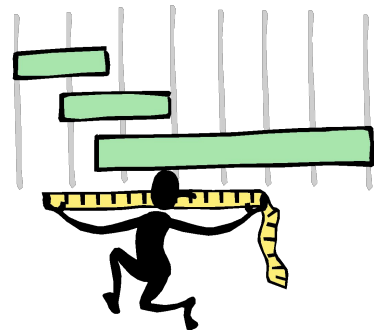
Analysis of Algorithms

- Input size (number of elements in the input)
 - size of an array
 - # of elements in a matrix
 - # of bits in the binary representation of the input
 - vertices and edges in a graph

Why study algorithms and performance?

- Algorithms help us to understand *scalability*.
- Performance often draws the line between what is feasible and what is impossible.
- Algorithmic mathematics provides a *language* for talking about program behavior.
- Performance is the *currency* of computing.

Measure Actual Running Time?



- We can measure the actual running time of a **program**
 - Use **stopwatch** or insert timing code into program
- However, actual running time is not meaningful when **comparing two algorithms**
 - Coded in different languages
 - Using different data sets
 - Running on different computers



Should we give up trying to measure?

Counting Operations

- What we can control?

- ❑ Count operations instead of time

- ❑ The number of operations and count them.

- ❑ Large input, more operations

- ❑ Small input, less operations

- ❑ Focus on how performance scales (how the number of operations grows as our input grows)

- ❑ Go beyond input size



(how the algorithm performs in all sort of situations.. How the internal structure of the input)

Example: Counting Operations

- How many operations are required?

```
for (int i = 1; i <= n; i++) {  
    perform 100 operations; // A  
    for (int j = 1; j <= n; j++) {  
        perform 2 operations; // B  
    }  
}
```

```
for(int i=1; i<=5; i++)  
{  
    printf("Hello - \n");  
    for(int j=1; j<=2; j++)  
    {  
        printf("CSE JU\n");  
    }  
}
```

Example: Counting Operations

- How many operations are required?

```
for (int i = 1; i <= n; i++) {
    perform 100 operations; // A
    for (int j = 1; j <= n; j++) {
        perform 2 operations; // B
    }
}
```

$$\begin{aligned}
 \text{Total Ops} &= \begin{matrix} \text{A} \\ \text{B} \end{matrix} + \begin{matrix} n & n & n \\ \sum_{i=1}^n 100 + \sum_{i=1}^n (\sum_{j=1}^n 2) \end{matrix} \\
 &= 100n + \sum_{i=1}^n 2n = 100n + 2n^2 = 2n^2 + 100n
 \end{aligned}$$

Example: Counting Operations

- Knowing the number of operations required by the algorithm, we can state that
 - Algorithm X takes $2n^2 + 100n$ operations to solve problem of size n
- If the time t needed for one operation is known, then we can state
 - Algorithm X takes $(2n^2 + 100n)t$ time units

Example: Counting Operations

- However, time t is directly dependent on the factors mentioned earlier
 - e.g. different languages, compilers and computers
- Instead of tying the analysis to actual time t , we can state
 - Algorithm X takes time that is **proportional to $2n^2 + 100n$** for solving problem of size n

Approximation of Analysis Results

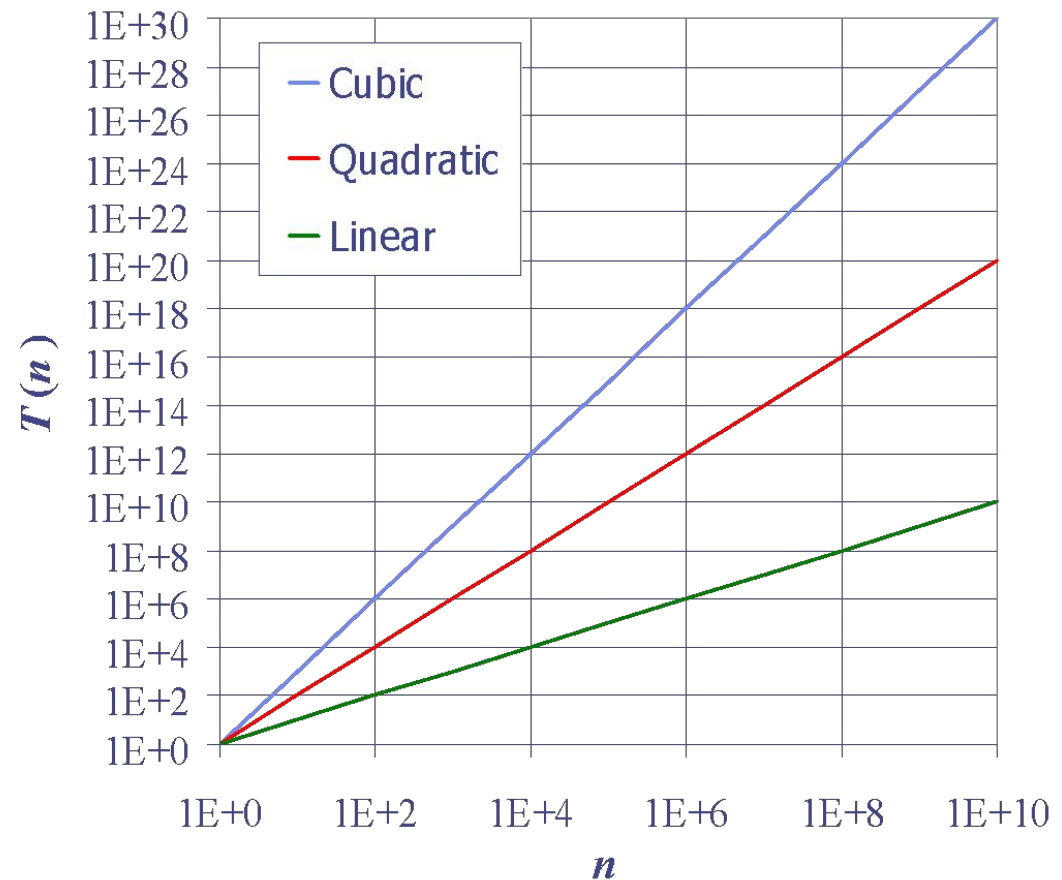
- Suppose the time complexity of
 - Algorithm *A* is $3n^2 + 2n + \log n + 1/(4n)$
 - Algorithm *B* is $0.39n^3 + n$
- Intuitively, we know Algorithm *A* will outperform *B*
 - When solving larger problem, i.e. larger n
- The **dominating term** $3n^2$ and $0.39n^3$ can tell us approximately how the algorithms perform
- The terms n^2 and n^3 are even simpler and preferred
- These terms can be obtained through **asymptotic analysis**

Asymptotic Analysis

- Asymptotic analysis is an analysis of algorithms that focuses on
 - Analyzing problems of large input size
 - Consider only the leading term of the formula
 - Ignore the coefficient of the leading term

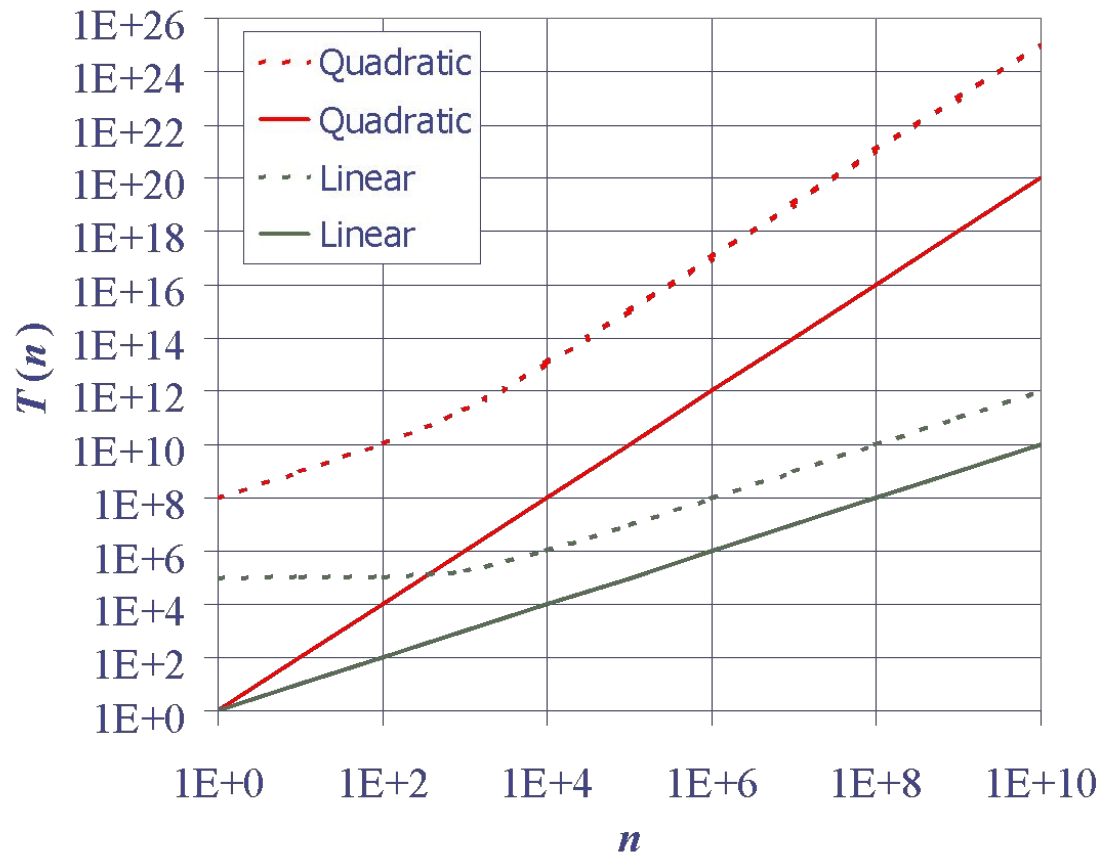
Why Choose Leading Term?

- Growth rates of functions:
 - Linear $\approx n$
 - Quadratic $\approx n^2$
 - Cubic $\approx n^3$
- In a log-log chart, the slope of the line corresponds to the growth rate of the function



Why Choose Leading Term?

- The growth rate is not affected by
 - constant factors or
 - lower-order terms
- Examples
 - $10^2n + 10^5$ is a linear function
 - $10^5n^2 + 10^8n$ is a quadratic function



Why Choose Leading Term?

- Lower order terms contribute lesser to the overall cost as the input grows larger

n	$f(n) = n^3$	$f(n) = n^3 + n^2$	$f(n) = n^3 + n^2 + 20n$
1	1	2	22
10	1,000	1,100	1,300
1,000	1,000,000,000	1,001,000,000	1,001,020,000
1,000,000	1.0×10^{18}	1.000001×10^{18}	1.000001×10^{18}

Why Choose Leading Term?

- Lower order terms contribute lesser to the overall cost as the input grows larger

n	$f(n) = n^3$	$f(n) = n^3 + n^2$	$f(n) = n^3 + n^2 + 20n$
1	1	2	22
10	1,000	1,100	1,300
1,000	1,000,000,000	1,001,000,000	1,001,020,000
1,000,000	1.0×10^{18}	1.000001×10^{18}	1.000001×10^{18}

Why Choose Leading Term?

- Lower order terms contribute lesser to the overall cost as the input grows larger

n	$f(n) = n^3$	$f(n) = n^3 + n^2$	$f(n) = n^3 + n^2 + 20n$
1	1	2	22
10	1,000	1,100	1,300
1,000	1,000,000,000	1,001,000,000	1,001,020,000
1,000,000	1.0×10^{18}	1.000001×10^{18}	1.000001×10^{18}

Why Choose Leading Term?

- Lower order terms contribute lesser to the overall cost as the input grows larger

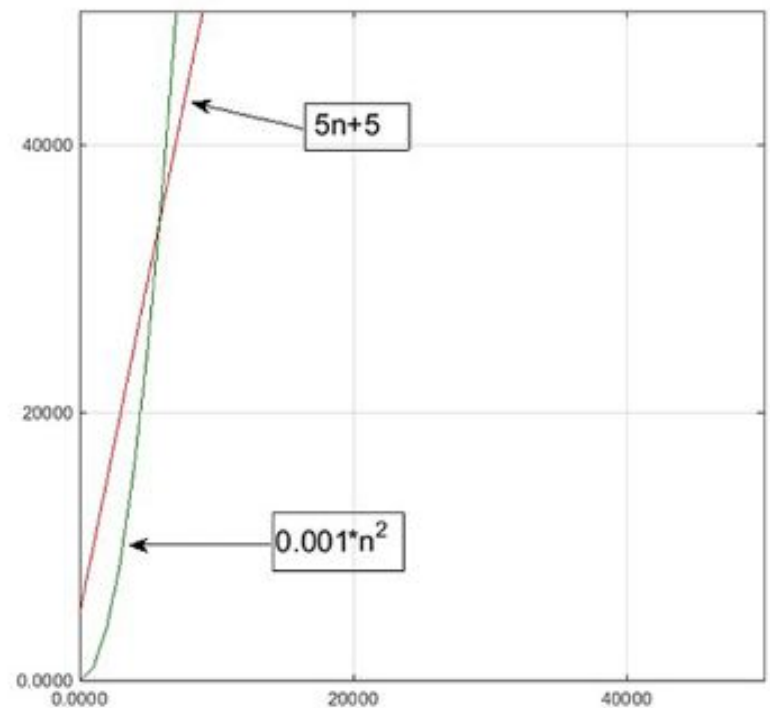
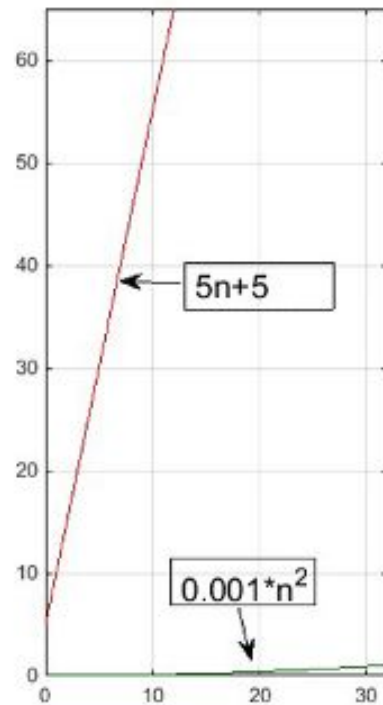
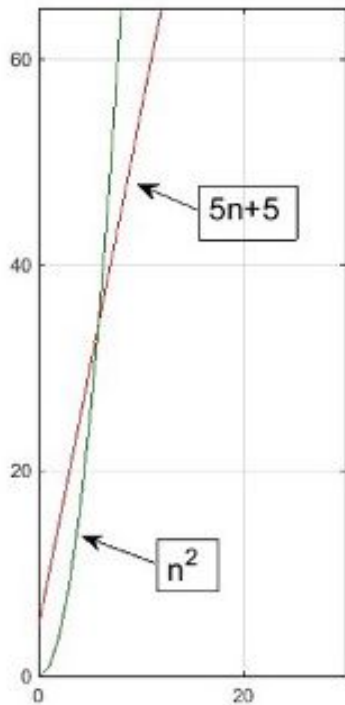
n	$f(n) = n^3$	$f(n) = n^3 + n^2$	$f(n) = n^3 + n^2 + 20n$
1	1	2	22
10	1,000	1,100	1,300
1,000	1,000,000,000	1,001,000,000	1,001,020,000
1,000,000	1.0×10^{18}	1.000001×10^{18}	1.000001×10^{18}

Why Choose Leading Term?

- Lower order terms contribute lesser to the overall cost as the input grows larger

n	$f(n) = n^3$	$f(n) = n^3 + n^2$	$f(n) = n^3 + n^2 + 20n$
1	1	2	22
10	1,000	1,100	1,300
1,000	1,000,000,000	1,001,000,000	1,001,020,000
1,000,000	1.0×10^{18}	1.000001×10^{18}	1.000001×10^{18}

Why ignore the coefficient of the leading term



How Do We Analyze Running Time?

- We need to define an objective measure.
 - Count the number of statements executed
 - Associate a "cost" with each statement and find the "total cost" by finding the total number of times each statement is executed.
 - **Not good**: number of statements vary with the programming language as well as the style of the individual programmer; therefore it can't be an objective measure of running time. For e.g. following two algorithms do the same task using different programming style.

Algorithm 1

Time (micro sec)

```

arr[0] = 0;      c1
arr[1] = 0;      c1
arr[2] = 0;      c1
...
arr[N-1] = 0;    c1
-----f
(n) = c1 + c1 + ... + c1 = c1N
    
```

Algorithm 2

Time (micro sec)

```

      c1  c2  c3
for(i=0; i<N; i++)  c1+c2(N+1)+c3N
    arr[i] = 0;      c1N
-----
g(n) = c1 + c2(N+1) + c3N + c1N = (c1 + c2 + c3)N + (c1 + c2)
    
```

Observation: For very large values of N, execution time of both programs is similar. Thus we can say that both of them have roughly the same running time.

Ideal Solution to Express running time

- Express runtime as a function of the input size n (i.e., $f(n)$) in order to understand how $f(n)$ grows with n
- *and* count only the most significant term of $f(n)$ and ignore everything else (because those won't affect running time much for very large values of n).
- Thus the running times (also called **time complexity**) of the programs of the previous slide becomes:
 - $f(N) = c_1 N \approx N * (\text{some constant})$
 - $g(N) = (c_1 + c_2 + c_3)N + (c_1 + c_2) \approx N * (\text{some constant})$
- Thus both these functions grows linearly with N and as such both have the same running time (specifically, linear running time). We say that both of them are **$O(N)$** functions, which means that, the running time of each of these algorithms is a constant multiple of N (we ignore the value of the constant).
- We compare running times of different functions in an *asymptotic* manner (i.e., we check if $f(n) > g(n)$ for very large values of n). That's why, the task of computing **time complexity** (of an algorithm) is also called **asymptotic analysis**.

Intuition behind Asymptotic Analysis

- Consider the example of buying *elephants* and *goldfish*:
Cost: cost_of_elephants + cost_of_goldfish
Cost $\sim$ cost_of_elephants (approximation)
- The low order terms, as well as constants in a function are relatively insignificant for **large** n

$$f(n) = 6n + 4 \approx n$$

$$g(n) = n^4 + 100n^2 + 10n + 50 \approx n^4$$

i.e., we say that $n^4 + 100n^2 + 10n + 50$ and n^4 have the same **rate of growth**. However, $f(n)$ and $g(n)$ have different rate of growths.

The Purpose of Asymptotic Analysis

- To estimate how long a program will run.
- To estimate the largest input that can reasonably be given to the program.
- To compare the efficiency of different algorithms.
- To help focus on the parts of code that are executed the largest number of times.
- To choose an algorithm for an application.

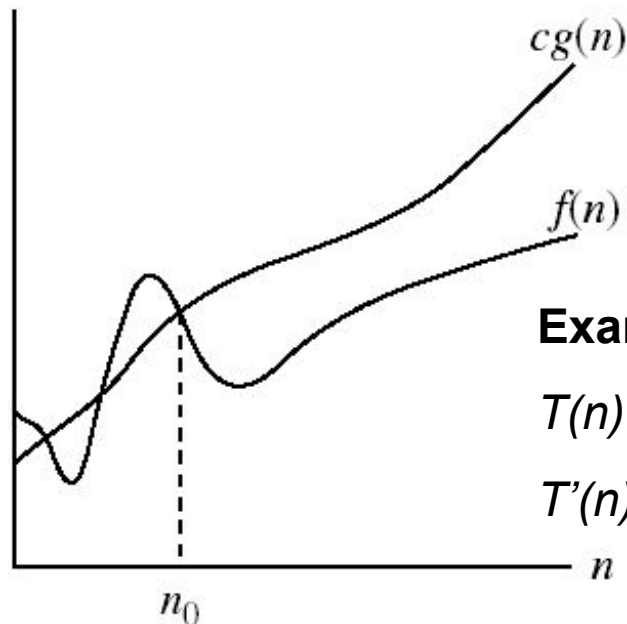
Asymptotic Notations

- O notation: asymptotic “upper bound”:
- Ω notation: asymptotic “lower bound”:
- Θ notation: asymptotic “tight bound”:

Asymptotic Notations

- O*-notation (*Big Oh*)

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}.$



$O(g(n))$ is the set of functions with smaller or same order of growth as $g(n)$

Examples:

$T(n) = 3n^2 + 10n \lg n + 8$ is $O(n^2)$, $O(n^2 \lg n)$, $O(n^3)$, $O(n^4)$, ...

$T'(n) = 52n^2 + 3n^2 \lg n + 8$ is $O(n^2 \lg n)$, $O(n^3)$, $O(n^4)$, ...

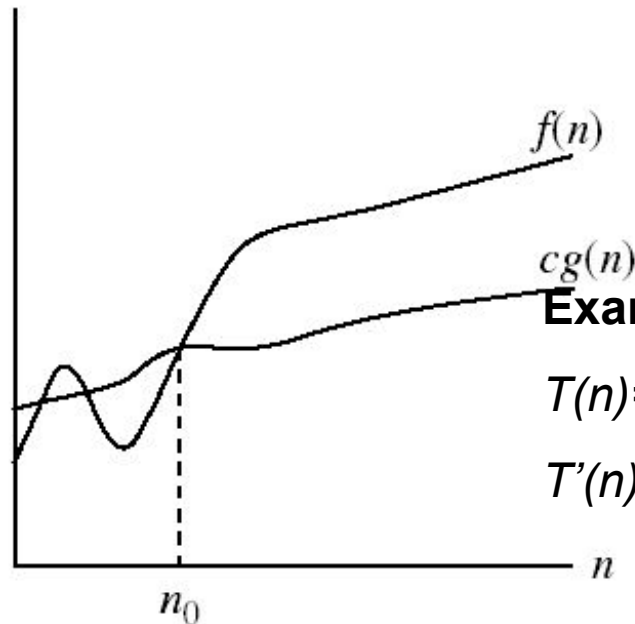
$g(n)$ is an *asymptotic upper bound* for $f(n)$.

Loose upper bounds

Asymptotic Notations

- Ω - notation (*Big Omega*)

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$.



$\Omega(g(n))$ is the set of functions with larger or same order of growth as $g(n)$

Examples:

$T(n) = 3n^2 + 10n \lg n + 8$ is $\Omega(n^2)$, $\Omega(n \lg n)$, $\Omega(n)$, $\Omega(\lg n)$, $\Omega(1)$

$T'(n) = 52n^2 + 3n^2 \lg n + 8$ is $\Omega(n^2 \lg n)$, $\Omega(n^2)$, $\Omega(n)$, ...

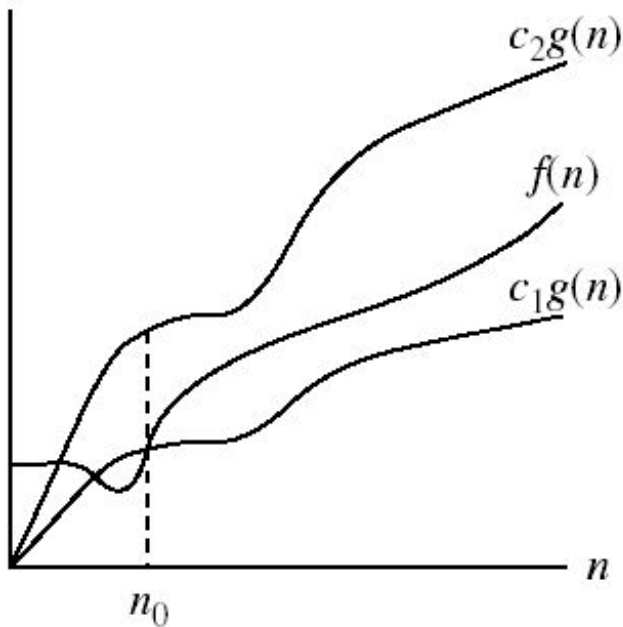
$g(n)$ is an *asymptotic lower bound* for $f(n)$.

Asymptotic Notations

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that}$
 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\} .$

$\Theta(g(n))$ is the set of functions with the same order of growth as $g(n)$

* $f(n)$ is both $O(g(n))$ & $\Omega(g(n)) \leftrightarrow f(n)$ is $\Theta(g(n))$



Examples:

$T(n) = 3n^2 + 10n \lg n + 8$ is $\Theta(n^2)$

$T'(n) = 52n^2 + 3n^2 \lg n + 8$ is $\Theta(n^2 \lg n)$

$g(n)$ is an *asymptotically tight bound* for $f(n)$.

Big O

- Informally, $O(g(n))$ is the set of all functions with a smaller or same order of growth as $g(n)$, within a constant multiple
- If we say $f(n)$ is in $O(g(n))$, it means that $g(n)$ is an **asymptotic upper bound** of $f(n)$
 - Intuitively, it is like $f(n) \leq g(n)$
- What is $O(n^2)$?
 - The set of all functions that grow slower than or in the same order as n^2

Examples

- Show that $30n+8$ is $O(n)$.
 - Show $\exists c, n_0: 30n+8 \leq cn, \forall n \geq n_0$.
 - Let $c=31, n_0=8$
 $cn = 31n = 30n + n \geq 30n+8,$
 - so $30n+8 \leq cn$.

Big-O example, graphically

- Note $30n+8$ isn't less than n *anywhere* ($n>0$).
- It isn't even less than $31n$ *everywhere*.
- But it *is* less than $31n$ everywhere to the right of $n=8$.

